



Universidade Federal de Sergipe · Programa de Pós-Graduação em Ciência da Computação (PROCC)
Projeto e Análise de Algoritmos (PROCC0083), turma T01, período 2026.2 · Prof. Glauco de Figueiredo Carneiro
Módulo I: Fundamentos e Eficiência na Era dos Dados · Atividade 1 (AV1) · Relatório técnico

Recuperação de contexto para IA generativa em materiais educacionais abertos

Corretude, eficiência e qualidade de três estratégias de ordenação e busca sobre o livro *Pense Python*

João Cosme Sena Sá · Eduardo Henrique do Lago Silva · Helena Carvalho Leal
Hernandison da Silva Bispo · Nadianne Maria dos Santos Galvão · Rafael Takeguma Goto

Tema	8, materiais educacionais abertos
Corpus	<i>Pense Python</i> , tradução livre da 3ª edição de <i>Think Python</i> (Allen B. Downey; trad. Rodrigo Castelan Carlson)
Repositório	https://github.com/joaocss/PAA_UFS_2026_2_SenaSa_Joao_Bispo_Hernandison_Galvao_Nadianne_Goto-Rafael_Leal_Helena_Silva_Eduardo
Licença · versão	MIT · commit 68fa986 do main; tag v1.0.0 na entrega
Vídeo da atividade	Gravações individuais: João, https://www.loom.com/share/aab4b4cd6a934bb892fd6c4bb73bcd85 ; Nadianne, https://drive.google.com/file/d/1WWWhyEl1k9qLuE8Sod03WE9qJ3bNDWx3Y/view?usp=drive_link ; Rafael, https://drive.google.com/file/d/13jiWmtcPA6ZGqowYDyxqEHd5966waPa8/view?usp=sharing . A URL do vídeo consolidado substitui esta lista em README.md e VIDEO.md.

São Cristóvão, SE · 23 de setembro de 2026 · Apresentação em 24/09/2026

Sumário

1. Identificação da equipe	11. Metodologia experimental
2. Tema, corpus e fonte	12. Resultados e gráficos
3. Repositório, licença, versão e data de acesso	13. Discussão de escalabilidade
4. Definição formal do problema	14. Relação com IA generativa e RAG
5. Representação dos dados e estratégia de chunking	15. Limitações e ameaças à validade
6. Algoritmos e pseudocódigo	16. Declaração de uso de IA generativa
7. Justificativa de correteude	17. Contribuição individual
8. Análise no modelo RAM	18. Vídeo da atividade
9. Melhor, pior e caso médio	19. Referências
10. Recorrências	

Este relatório foi montado a partir dos documentos versionados no repositório: docs/RESULTADOS.md, docs/CORRETEUDE.md, docs/PROTOCOLO_EXPERIMENTAL.md, docs/CONTRATOS.md, docs/USO_DE_IA.md, docs/CONTRIBUICOES.md, data/README.md e das tabelas e figuras de experimentos/. Todos os números vêm de experimentos/brutos/execucoes.jsonl, gerado por scripts/rodar_experimentos.py.

1. Identificação da equipe

Equipe de seis discentes matriculados em Projeto e Análise de Algoritmos (PROCC0083), turma T01, período 2026.2, na modalidade de trabalho em equipe (5 a 7 integrantes).

Integrante	Frete principal
João Cosme Sena Sá	Dados de teste, pacote e testes, protocolo e execução dos experimentos, índice invertido e busca binária (C2), entrega
Eduardo Henrique do Lago Silva	Experimentos, tabelas, gráficos e baseline de referência (C4)
Helena Carvalho Leal	Consultas, julgamentos de relevância, métricas, relação com IA generativa e limitações
Hernandison da Silva Bispo	Ingestão, normalização e fragmentação; manutenção do relatório
Nadianne Maria dos Santos Galvão	Correteude, modelo RAM, complexidade e recorrências
Rafael Takeguma Goto	Busca linear, Insertion Sort, Merge Sort, pontuação e consulta (C1 e C3)

A tabela de contribuição individual com evidências (commits, arquivos, execuções) está na Seção 17 e em docs/CONTRIBUICOES.md. A revisão de código, da prova e dos resultados é responsabilidade coletiva.

2. Tema, corpus e fonte

O tema é o de número 8 do enunciado, materiais educacionais abertos, com a pergunta de recuperação "quais são os conceitos-chave de um tópico?" instanciada em 30 consultas de estudante sobre o livro. O corpus é *Pense Python*, tradução livre da 3ª edição de *Think Python*, de Allen B. Downey, traduzida por Rodrigo Castelan Carlson: conteúdo didático público, sem dados pessoais ou sigilosos.

Campo	Registro
Fonte	https://rodrigocarlon.github.io/PensePython3ed/ e https://github.com/rodrigocarlon/PensePython3ed
Commit de referência	cfde2c450bf4f2ea65326da88e9968f400597e92 (branch main)
Data de acesso	02/09/2026 (aquisição registrada em data/corpus_manifest.csv)
Recorte e formato	21 notebooks Jupyter (capitulos/*.ipynb): prefácio, introdução ao Jupyter e capítulos 1 a 19; as pastas brancos, solucoes e capitulos/teste ficam de fora
Tamanho auditado	817.578 bytes, 2.789 células, 73.867 palavras; SHA-256 do recorte 20106262...68dc5
Extração	2.737 chunks em data/processed/chunks.jsonl; SHA-256 2af8c533...63cc4, conferido em reprodução independente
Idioma	Português brasileiro, com estrangeirismos técnicos do Python e do Jupyter
Licenças	Texto CC BY-NC-SA 4.0; códigos do livro MIT; código da equipe MIT
Dados removidos	Saídas de execução das células, descartadas para não introduzir texto derivado de execução; nenhum dado sensível

Como a licença do texto é não comercial e com compartilhamento pelas mesmas condições, o texto não é redistribuído como parte do código MIT da equipe: scripts/baixar_corpus.py baixa o material no commit fixado e o manifesto de hashes permite conferir que o download reproduz o mesmo recorte. Trechos citados preservam a atribuição a autor e tradutor.

3. Repositório, licença, versão e data de acesso

Item	Registro
URL do repositório	https://github.com/joaocss/PAA_UFS_2026_2_SenaSa_Joao_Bispo_Hernandison_Galvao_Nadianne_Goto_Rafael_Leal_Helena_Silva_Eduardo
Licença do código	MIT (arquivo LICENSE)
Commit / tag	commit 68fa986 do main em 23/09/2026; tag v1.0.0 criada na entrega; CITATION.cff

Data de acesso ao corpus	02/09/2026
Ambiente	Python 3.11 ou superior; dependências em <code>requirements.lock</code> (pytest 8.3.4, scikit-learn 1.6.1, matplotlib 3.11.2, numpy 2.2.2); C1, C2 e C3 não usam biblioteca externa
Reprodução	<code>python scripts/reproduzir_tudo.py</code> : download, chunks, testes, experimentos, tabelas, figuras, ingestão e sensibilidade; cerca de 15 a 20 minutos

O repositório não contém chaves, senhas, tokens ou dados pessoais. O corpus baixado e os dados intermediários ficam fora do versionamento por `.gitignore`; os dados brutos, as tabelas e as figuras dos experimentos são versionados por serem entregáveis.

4. Definição formal do problema

O protótipo é um recuperador de contexto: dada uma consulta, devolve os trechos mais relevantes do corpus, ordenados por relevância. Não é um chatbot; o interesse está nas operações algorítmicas anteriores à geração de texto.

4.1 Entradas e saída

- **Corpus** $C = \{c_1, \dots, c_N\}$: sequência de N chunks. Cada chunk tem `chunk_id` textual único, `source_order` inteiro único em $0..N-1$ (posição no livro, usada no desempate), o texto normalizado e a origem (notebook, capítulo, seção, célula).
- **Consulta** q : cadeia em português; após normalização vira um multiconjunto de m termos.
- $k \in \mathbb{N}$: número máximo de trechos a devolver.
- **Saída** R : sequência de até k chunks de C , sem repetição, ordenada de forma não crescente pela função de relevância, com desempate explícito.

4.2 Função de relevância e ordem total

A pontuação é tf-idf lexical, definida pela equipe (contrato 02 de `docs/CONTRATOS.md`) e usada de forma idêntica nas três configurações obrigatórias. Para um termo t da consulta e um chunk c , com $f(t, c)$ ocorrências de t em c , N chunks e $df(t)$ chunks que contêm t :

$$\begin{aligned} \text{tf}(t, c) &= 1 + \ln f(t, c), \text{ se } f(t, c) > 0; \text{ senão } 0 \\ \text{idf}(t) &= \ln((N + 1) / (df(t) + 1)) + 1 \\ \text{escore}(q, c) &= \sum_t \text{qt}(t) \cdot \text{tf}(t, c) \cdot \text{idf}(t) \quad (t \text{ sobre os termos distintos de } q; \text{qt} = \text{ocorrências de } t \text{ em } q) \end{aligned}$$

As formas suavizadas são as mesmas do `TfidfVectorizer` do scikit-learn, o que mantém limpa a comparação com a configuração de referência e nunca zera um termo presente em todos os chunks. A ordem total $\leq$ usada na ordenação é: $c_a \leq c_b \Leftrightarrow \text{escore}(q, c_a) > \text{escore}(q, c_b) \vee (\text{escores iguais} \wedge \text{source_order_a} < \text{source_order_b})$. O desempate por posição no livro torna o resultado independente da ordem de entrada e da configuração.

4.3 Pré-condições, pós-condições, casos de borda e hipótese sobre os dados

Categoria	Especificação
Pré-condições	$N \geq 0$; $k \geq 0$; todo escore é real finito; os <code>source_order</code> são únicos; $\leq$ é uma ordem total sobre os chunks
Pós-condições	Seja n' o número de chunks com <code>escore</code> > 0 . Então $ R = \min(k, n')$; todo elemento de R pertence a C e é único; R está ordenado por $\leq$; não existe chunk fora de R com relevância maior, por $\leq$, que algum elemento de R
Casos de borda	consulta vazia após normalização $\rightarrow R = \{\}$; $k = 0 \rightarrow \{\}$; $k > n' \rightarrow$ devolve os n' com <code>escore</code> > 0 ; nenhum termo casa $\rightarrow \{\}$; empates resolvidos por <code>source_order</code> ; termo fora do vocabulário ignorado. Todos cobertos por testes em <code>tests/</code>
Hipótese sobre os dados	A vantagem do índice invertido (C2) supõe esparsidade: os termos da consulta ocorrem em poucos chunks ($df \ll N$), de modo que o conjunto de candidatos é bem menor que N . A correteza não depende dessa hipótese, só a eficiência. A hipótese foi testada e não se confirmou neste corpus (Seções 12 e 13): a mediana de candidatos é 51 % dos chunks

5. Representação dos dados e estratégia de chunking

5.1 Normalização

Deliberadamente simples, para poder ser auditada. Na ingestão (`fragmentacao.py`), as células Markdown passam por NFKC, minúsculas, separação controlada de pontuação e compactação de espaços, com acentos preservados no texto guardado no chunk; as células de código guardam a fonte intacta. Na pontuação (`preprocessing.normalize`), consulta e chunk passam pela mesma função: minúsculas, remoção de acentos (NFD sem marcas combinantes) e separação em tokens alfanuméricos; por isso "função" e "funcao" produzem o mesmo escore. Stopwords não são removidas: em português isso apagaria termos funcionais que mudam o sentido de consultas como "diferença entre while e for". Saídas de execução são descartadas.

5.2 Segmentação

A segmentação respeita notebook, título e seção, nunca mistura célula de texto com célula de código e usa janelas de 256 tokens com sobreposição de 32 dentro de cada célula (token é uma palavra após a normalização). Como a maior parte das células é menor que a janela, o corte por célula domina: o corpus inteiro rende **2.737 chunks**, com mediana de 13 tokens, e só 0,8 % deles vêm de células divididas em mais de uma janela. A estimativa inicial de cerca de 330 chunks, feita na reserva do corpus supondo janelas contínuas, não se confirmou. As configurações 128/16 e 512/64 entram no experimento de sensibilidade (Seção 12.5).

5.3 Representações computacionais

- **Chunk** (contrato 01): registro com `chunk_id`, `source_order`, texto, tipo, arquivo, capítulo, seção, célula e offsets, um objeto JSON por linha.
- **Estatísticas do corpus**: N e df por termo, calculados uma vez antes das consultas.
- **Índice invertido** (C2): vocabulário como vetor ordenado de termos e, para cada termo, a lista crescente das posições dos chunks que o contém (postings). O dicionário usado na construção é descartado; a consulta localiza termos só por busca binária.
- **Referência C4**: matriz esparsa tf-idf do scikit-learn com norma L2; escore por cosseno.

6. Algoritmos e pseudocódigo

Os cinco algoritmos obrigatórios (§5.2 do enunciado) são cobertos assim: busca linear na pontuação de todos os chunks; ordenação por Insertion Sort e Merge Sort implementados pela equipe; busca binária no vocabulário do índice invertido; divisão e conquista pelo próprio Merge Sort; comparação com biblioteca pelo tf-idf do scikit-learn, usado apenas como referência (C4). Os pseudocódigos abaixo descrevem o código real de `src/paa_context/`.

6.1 Busca linear com pontuação (C1 e C3)

```
função PontuarLinear(q, C, estatísticas):
    termos_q ← Contagem(normalize(q))           # m termos distintos
    candidatos ← {}
    para cada chunk c em C:                      # N iterações
        termos_c ← Contagem(normalize(c.texto))  # O(n) para um chunk de n tokens
        s ← Σ_{t ∈ termos_q, f = termos_c[t] > 0} qtf(t) · (1 + ln f) · idf(t)
        se s > 0: candidatos.anexar((c, s))
    retorne candidatos                          # na ordem do livro
```

6.2 Insertion Sort (C1)

```
função InsertionSort(A, ≤):
    para i ← 1 até |A|-1:
        chave ← A[i]; j ← i - 1
        enquanto j ≥ 0 e chave ≤ A[j]:          # comparação contada
            A[j+1] ← A[j]; j ← j - 1            # deslocamento contado como troca
        A[j+1] ← chave
    retorne A
```

6.3 Merge Sort (C3, e ordenação dos candidatos em C2)

```
função MergeSort(A, ≤):
    se |A| ≤ 1: retorne cópia de A
    meio ← ⌊|A| / 2⌋
    E ← MergeSort(A[0..meio-1]); D ← MergeSort(A[meio..])
    retorne Merge(E, D, ≤)

função Merge(E, D, ≤):
    O ← {}; i ← 0; j ← 0
    enquanto i < |E| e j < |D|:
        se E[i] ≤ D[j]: O.anexar(E[i]); i ← i+1    # empate nunca ocorre entre ids distintos
        senão: O.anexar(D[j]); j ← j+1
    O.anexar(E[i..]); O.anexar(D[j..])
    retorne O
```

6.4 Índice invertido e busca binária (C2)

```
função ConstruirÍndice(C):
    por_termo ← dicionário vazio
    para cada posição p, chunk c em C:
        para cada termo t distinto em normalize(c.texto): por_termo[t].anexar(p)
    vocab ← ordenar(chaves de por_termo)           # uma vez, fora da consulta
    postings[i] ← por_termo[vocab[i]]
    retorne (C, vocab, postings)

função BuscaBinária(vocab, t):                    # vocab ordenado pela mesma chave
    lo ← 0; hi ← |vocab| - 1
    enquanto lo ≤ hi:
        mid ← ⌊(lo + hi) / 2⌋
        se vocab[mid] = t: retorne mid
        senão se vocab[mid] < t: lo ← mid + 1
        senão: hi ← mid - 1
    retorne -1

função ConsultarÍndice(q, índice, estatísticas):
    posições ← {}
    para cada termo t distinto em normalize(q):    # m termos
        i ← BuscaBinária(vocab, t)                # O(log V)
        se i ≥ 0: posições ← UniãoOrdenada(posições, postings[i])  # dois ponteiros
    candidatos ← ((c, escore(q, c)) para p em posições, c = C[p], se escore > 0)  # repontua só os candidatos
    retorne MergeSort(candidatos, ≤)
```

6.5 Seleção dos k melhores

```
função TopK(ordenados, k): retorne os primeiros min(k, |ordenados|) elementos
```

Como a lista já está totalmente ordenada por $\leq$, os primeiros $\min(k, n)$ elementos são os k mais relevantes (corolário da Seção 7). A referência C4 chama `TfidfVectorizer` e ordena os cossenos com `numpy.argsort`; a equipe impõe só o corte por `escore > 0` e o desempate.

7. Justificativa de corretude

A prova cobre o Merge Sort, núcleo de C3 e da ordenação de C2. Texto integral de `docs/CORRETUDE.md`, de Nadianne Maria dos Santos Galvão, conferido contra `merge_sort.py` e `sort_key.py`.

O que se deseja provar

Dada uma lista finita de registros válidos e uma ordem total definida por pontuação decrescente e, em caso de empate, identificador de origem crescente, o Merge Sort implementado pela equipe termina e devolve uma permutação da entrada ordenada por essa relação. Como consequência, os primeiros $\min(k, n)$ elementos do resultado são exatamente os k mais relevantes.

Hipóteses

Para que a justificativa de corretude seja válida, consideram-se as seguintes condições:

1. A entrada do algoritmo é uma lista finita de candidatos.
2. Cada candidato possui um `score` numérico válido, definido e comparável.
3. Cada candidato possui um valor de `source_order` válido, utilizado como critério de desempate quando dois candidatos apresentam o mesmo `score`.
4. A regra de ordenação considera primeiro o maior `score` e, em caso de empate, o menor `source_order`.
5. A relação de comparação utilizada é consistente e transitiva. Isso significa que, se um elemento deve aparecer antes de um segundo elemento, e esse segundo deve aparecer antes de um terceiro, então o primeiro também deve aparecer antes do terceiro.
6. O procedimento de merge recebe duas listas previamente ordenadas segundo essa mesma regra.

Essas hipóteses garantem que os elementos possam ser comparados de forma consistente durante a execução do algoritmo e que exista uma ordem bem definida para o resultado final.

Prova por indução

Para demonstrar a corretude do Merge Sort implementado no projeto, utiliza-se uma prova por indução sobre o tamanho da lista de candidatos. Esse tipo de raciocínio é compatível com a estrutura recursiva do algoritmo: o problema é dividido em subproblemas menores até atingir casos triviais e, em seguida, os resultados são combinados. Essa organização segue o paradigma de divisão e conquista apresentado por Cormen et al. (2001, Seção 2.3).

Seja $P(n)$ a seguinte proposição:

Para toda lista válida de tamanho n , o algoritmo `merge_sort` termina e retorna uma permutação da entrada ordenada segundo a regra definida por `sort_key`.

A função `sort_key` utiliza como critério $(-\text{score}, \text{source_order})$, isto é, maior `score` primeiro e, em caso de empate, menor `source_order`.

Caso base

Para listas de tamanho $n \leq 1$, não é necessário realizar nenhuma divisão adicional. Uma lista vazia ou contendo apenas um elemento já está ordenada por definição.

Na implementação da equipe, quando o tamanho da entrada é menor ou igual a um, a função retorna diretamente uma cópia da própria lista.

Portanto, para $n = 0$ e $n = 1$, o algoritmo:

- termina;
- preserva os elementos da entrada;
- retorna uma lista já ordenada.

Assim, a proposição $P(n)$ é verdadeira nos casos base.

Hipótese de indução

Assume-se que a proposição seja verdadeira para todas as listas de tamanho menor que n .

Em outras palavras, considera-se que, para qualquer lista válida de tamanho $j < n$, o algoritmo `merge_sort`:

- termina;
- preserva todos os elementos da entrada;
- retorna esses elementos ordenados segundo `sort_key`.

Essa hipótese permite analisar o comportamento do algoritmo para uma lista de tamanho n .

Passo indutivo

Considere agora uma lista válida de tamanho $n > 1$.

O algoritmo divide essa lista em duas partes menores, `left` e `right`. Como ambas possuem tamanho inferior a n , aplica-se a hipótese de indução: as chamadas recursivas `merge_sort(left)` e `merge_sort(right)` terminam e produzem duas listas corretamente ordenadas.

Na implementação da equipe, essas duas chamadas recursivas são executadas antes da etapa de combinação.

Em seguida, o procedimento `_merge` combina as duas listas já ordenadas. A cada comparação, é selecionado o elemento que deve aparecer primeiro segundo `sort_key`, mantendo a ordem definida pelo projeto.

Como todos os elementos das duas metades são copiados para a lista resultante, sem alteração do conjunto de elementos da entrada, o resultado final é uma permutação da lista original. Os elementos restantes de uma das metades, após o término do laço principal, também são adicionados ao resultado.

Portanto, se o algoritmo funciona corretamente para os dois subproblemas menores, então também funciona corretamente para a lista de tamanho n .

Assim, $P(n)$ é verdadeira.

Conclui-se, por indução, que o Merge Sort implementado pela equipe termina e retorna uma permutação da entrada ordenada segundo a relação definida por `sort_key`, para toda lista finita que satisfaça as hipóteses estabelecidas.

Invariante do merge

Para demonstrar que o procedimento `_merge` mantém a ordenação correta durante a combinação de duas listas já ordenadas, considera-se o seguinte invariante de laço:

Antes de cada iteração do laço principal, a lista `merged` contém exatamente os elementos já consumidos de `left` e `right`, na ordem correta segundo `sort_key`.

Além disso, todos os elementos já inseridos em `merged` aparecem antes dos elementos ainda não consumidos, de acordo com a mesma relação de ordenação.

Na implementação da equipe, `_merge` compara os elementos atuais das duas listas e acrescenta ao resultado aquele que deve aparecer primeiro segundo `sort_key`.

Inicialização

Antes da primeira iteração:

- `merged` está vazia;
- nenhum elemento de `left` foi consumido;
- nenhum elemento de `right` foi consumido.

Logo, o invariante é verdadeiro nesse momento, pois a lista `merged` contém exatamente os elementos já processados — nesse caso, nenhum — e está trivialmente ordenada.

A demonstração por inicialização, manutenção e término segue a estrutura de prova por invariante de laço apresentada por Cormen et al. (2001, Seção 2.1).

Manutenção

Suponha que o invariante seja verdadeiro no início de uma determinada iteração.

Como `left` e `right` já estão ordenadas, seus primeiros elementos ainda não consumidos são os próximos candidatos possíveis segundo a relação de ordenação utilizada no projeto.

O algoritmo compara esses dois elementos por meio de `sort_key` e acrescenta a `merged` aquele que deve aparecer primeiro. Em seguida, o índice correspondente é incrementado.

Dessa forma:

- nenhum elemento já inserido precisa ser reposicionado;
- `merged` continua ordenada;
- `merged` continua contendo exatamente os elementos já consumidos de `left` e `right`.

Portanto, o invariante é preservado após cada iteração.

Término

O laço principal termina quando todos os elementos de uma das duas listas foram consumidos.

Nesse momento, os elementos restantes pertencem apenas à outra lista. Como essa lista já está ordenada, seus elementos podem ser acrescentados ao final de `merged` sem violar a ordem estabelecida. A implementação realiza essa etapa por meio dos laços que copiam os elementos restantes de `left` ou `right`.

Ao término do procedimento:

- todos os elementos de `left` e `right` estão presentes em `merged`;
- nenhum elemento é perdido;
- nenhum elemento é duplicado;
- a lista resultante permanece ordenada segundo `sort_key`.

Assim, o invariante permite justificar que o procedimento `_merge` combina corretamente duas listas previamente ordenadas, produzindo uma única lista também ordenada.

Término

A prova de término considera separadamente a recursão do `merge_sort` e os laços do procedimento `_merge`.

Término das chamadas recursivas

Para uma lista com mais de um elemento, o `merge_sort` divide a entrada em duas partes menores.

Essas partes possuem tamanho estritamente menor que o tamanho da lista original. O processo de divisão continua até que sejam alcançadas listas de tamanho zero ou um, que correspondem ao caso base do algoritmo. Na implementação da equipe, quando `len(items) <= 1`, a função retorna diretamente uma cópia da lista.

Como o tamanho do problema diminui a cada chamada recursiva, não é possível ocorrer uma sequência infinita de divisões. Assim, as chamadas recursivas terminam.

Esse comportamento é compatível com a estratégia de divisão e conquista descrita por Cormen et al. (2001, Seção 2.3), na qual o problema é dividido em subproblemas menores até atingir casos simples.

Término do procedimento `_merge`

No procedimento `_merge`, dois índices são utilizados para percorrer as listas `left` e `right`.

A cada iteração do laço principal, pelo menos um desses índices é incrementado, indicando que um elemento foi consumido de uma das listas.

Como `left` e `right` são listas finitas, os índices não podem ser incrementados indefinidamente. O laço principal termina quando uma das listas é completamente consumida.

Em seguida, os elementos restantes da outra lista são copiados para o resultado por meio de laços que também avançam seus respectivos índices a cada iteração.

Portanto, todos os laços executados por `_merge` também terminam.

Dessa forma, como tanto as chamadas recursivas quanto os laços de combinação possuem progresso finito e alcançam uma condição de parada, conclui-se que o Merge Sort implementado pela equipe termina para toda entrada finita que satisfaça as hipóteses estabelecidas.

Limites do argumento

A prova apresentada garante propriedades relacionadas ao funcionamento do algoritmo de ordenação, mas não garante que todo o processo de recuperação de informação esteja semanticamente correto.

Em particular, a prova garante que:

- o `merge_sort` termina para toda entrada finita que satisfaça as hipóteses estabelecidas;
- os elementos da entrada são preservados durante a ordenação;
- nenhum candidato é perdido ou duplicado pelo procedimento de ordenação;
- a saída é ordenada segundo a regra definida por `sort_key`;
- o critério de desempate por `source_order` é respeitado.

A regra usada pelo projeto prioriza maior `score` e, em caso de empate, menor `source_order`. Essa relação é implementada diretamente pela função `sort_key`.

Entretanto, essa demonstração não garante que o valor de `score` represente perfeitamente a intenção do usuário. Um candidato pode estar corretamente ordenado segundo a função de relevância utilizada e, ainda assim, não ser o trecho semanticamente mais adequado para responder à consulta.

Também não se pode concluir, apenas pela correteza da ordenação, que:

- todos os chunks recuperados sejam semanticamente relevantes;
- o corpus contenha todas as informações necessárias para responder à consulta;
- a função de relevância escolhida seja a melhor possível;
- o primeiro resultado ordenado seja necessariamente o melhor do ponto de vista humano;
- uma etapa posterior de geração de resposta produza conteúdo correto.

Assim, a prova demonstra a correteza do processo de ordenação em relação à regra definida pelo sistema, mas não a correteza semântica de todo o processo de recuperação.

Em outras palavras, o algoritmo pode ordenar corretamente os candidatos segundo o `score` calculado, mesmo que esse `score` não represente de forma perfeita o que o usuário pretendia encontrar.

Casos em que a prova não se aplica

A prova não se aplica, por exemplo, nos seguintes casos:

- quando algum candidato possui `score` indefinido, inválido ou não comparável;
- quando o campo `source_order` está ausente ou não pode ser comparado;
- quando a relação de comparação deixa de ser consistente ou transitiva;
- quando o procedimento `_merge` recebe listas que não estão previamente ordenadas segundo `sort_key`;
- quando as chamadas recursivas deixam de produzir subproblemas menores;
- quando a implementação utiliza um critério de comparação diferente daquele definido em `sort_key`;
- quando a estrutura dos elementos de entrada não corresponde ao formato esperado pelo algoritmo.

Na implementação atual, a ordenação depende da chave `(-score, source_order)`. Portanto, qualquer alteração nessa regra exige uma nova análise de correteza, pois a relação de ordem considerada na prova também mudaria.

Da mesma forma, a prova do Merge Sort pressupõe que cada metade seja ordenada antes da chamada ao procedimento `_merge`. Essa condição é satisfeita pela implementação atual por meio das chamadas recursivas realizadas antes da combinação das duas partes.

Assim, a demonstração é válida para a implementação e para as condições analisadas neste trabalho, não podendo ser generalizada automaticamente para versões modificadas do algoritmo.

Testes que ancoram o argumento

A prova formal de correteza apresentada neste trabalho é baseada em indução, invariante de laço e argumento de término. Os testes automatizados não substituem essa demonstração, mas funcionam como evidência prática de que a implementação respeita as propriedades assumidas na prova.

Teste específico da relação de ordenação

Como parte desta análise, foi criado o arquivo:

```
tests/unit/test_sort_key.py
```

Esse arquivo testa diretamente a regra de ordenação definida em:

```
src/paa_context/sort_key.py
```

A função `sort_key` utiliza a chave:

```
(-score, source_order)
```

Essa regra estabelece que:

- candidatos com maior `score` devem aparecer primeiro;
- em caso de empate de `score`, o menor `source_order` deve aparecer primeiro.

Essa relação é a mesma utilizada ao longo da prova formal para definir quando um candidato deve aparecer antes de outro.

Foram implementados três testes específicos:

- `test_maior_score_vem_primeiro`: com escores 20,0 e 10,0, o item de escore 20,0 tem a chave menor e vem antes, mesmo tendo `source_order` maior.
- `test_empate_usa_menor_source_order`: com escores iguais a 10,0, o item de `source_order` 1 vem antes do de `source_order` 2.
- `test_ordem_transitiva`: com três itens de escores 30,0, 20,0 e 10,0, as três comparações entre pares são coerentes entre si.

A estabilidade e o desempate dentro das ordenações são exercitados em `tests/unit/test_insertion_sort.py` e `tests/unit/test_merge_sort.py` (casos `test_empate_*`), e a equivalência entre as três configurações em `tests/integration/test_equivalencia.py`. O mapa completo está em `tests/README.md`.

8. Análise no modelo RAM

8.1 Parâmetros

Símbolo	Significado	Valor no corpus inteiro
N	número de chunks	2.737
L	total de tokens do corpus	72.205
n	tokens de um chunk	mediana 13, máximo 256
m	termos distintos da consulta	mediana 13
V	termos distintos do vocabulário	5.028
P	candidatos com escore > 0 para a consulta	mediana 1.403 (51 % de N)
S	soma dos postings dos termos da consulta, $\sum_t df(t)$	2.565 na consulta Q01
k	resultados devolvidos	5 e 10
r	repetições experimentais	5

8.2 Operações elementares e custo por algoritmo

Contam-se como operações elementares: comparação de chaves, acesso a vetor ou lista, atribuição, incremento, chamada recursiva, consulta a tabela de dispersão e anexação a lista. Os contadores instrumentados registram comparações entre chaves e trocas (deslocamentos no Insertion Sort, cópias para a saída no merge).

- **Pontuação linear.** Para cada um dos N chunks, `relevance_score` reconstrói a contagem de termos do chunk ($\Theta(n)$) e percorre os m termos da consulta com consulta $O(1)$ à tabela. Custo $\Theta(N \cdot m + L)$. Declarar só $\Theta(N \cdot m)$, tratando a comparação como constante, esconde o termo dominante: no corpus inteiro $L = 72.205 > N \cdot m \approx 35.600$, e o tempo cresce linearmente com n a $N \cdot m$ fixo. Espaço extra $\Theta(P)$, a lista de candidatos.
- **Insertion Sort** sobre P candidatos: o laço externo executa $P-1$ vezes e o interno desloca, no pior caso, todos os já ordenados: $\sum i = P(P-1)/2$ comparações, mesma ordem de trocas. Espaço $\Theta(1)$, no lugar.
- **Merge Sort** sobre P candidatos: cada nível da recursão percorre os P elementos uma vez no merge (no máximo $P-1$ comparações e P cópias por nível) e há $\lceil \log_2 P \rceil$ níveis: $\Theta(P \log P)$ comparações; pior caso $P \log_2 P - P + 1$. Espaço $\Theta(P)$ para as listas auxiliares.
- **Busca binária** no vocabulário: $\Theta(\log V)$ comparações por termo; $\Theta(1)$ de espaço.
- **Consulta indexada (C2).** m buscas binárias, $\Theta(m \log V)$; união repetida das listas de postings com dois ponteiros, no total $O(m \cdot S)$; repontuação dos P candidatos, $\Theta(P \cdot (m + n))$; Merge Sort, $\Theta(P \log P)$. Custo $O(m \log V + m \cdot S + P(m + n) + P \log P)$. Construção do índice, feita uma vez: $\Theta(L + V \log V)$; espaço $\Theta(V + \sum_t df(t))$.

Etapas	C1 (linear + Insertion)	C2 (índice + binária + Merge)	C3 (linear + Merge)
Pontuação / seleção de candidatos	$\Theta(N \cdot m + L)$	$O(m \log V + m \cdot S + P(m + n))$	$\Theta(N \cdot m + L)$
Ordenação dos candidatos	$O(P^2)$	$\Theta(P \log P)$	$\Theta(P \log P)$
Seleção top-k	$\Theta(k)$	$\Theta(k)$	$\Theta(k)$
Espaço adicional na consulta	$\Theta(P)$	$\Theta(V + \sum df)$ do índice + $\Theta(P)$	$\Theta(P)$

Fatores fora do modelo RAM. Hierarquia de cache, E/S na leitura dos notebooks, alocação e coletor de lixo do Python, e a diferença entre código interpretado (C1 a C3) e rotinas vetorizadas em C do scikit-learn (C4). Por isso a Seção 12 mede tempo de parede além de contar operações, e os contadores são a evidência principal.

9. Melhor, pior e caso médio

Algoritmo	Melhor caso	Pior caso	Caso médio	Espaço
Busca/pontuação linear	$\Theta(N \cdot m + L)$	$\Theta(N \cdot m + L)$	$\Theta(N \cdot m + L)$	$\Theta(P)$
Insertion Sort (P candidatos)	$\Theta(P)$, já ordenado	$\Theta(P^2)$, invertido	$\Theta(P^2)$, $\approx P^2/4$ comparações	$\Theta(1)$
Merge Sort (P candidatos)	$\Theta(P \log P)$	$\Theta(P \log P)$	$\Theta(P \log P)$	$\Theta(P)$
Busca binária (por termo)	$\Theta(1)$	$\Theta(\log V)$	$\Theta(\log V)$	$\Theta(1)$
Consulta via índice (C2)	$\Theta(m \log V)$, termos ausentes ou raros	$O(m \log V + m \cdot N + N(m + n) + N \log N)$, termos em todos os chunks	depende de S e P; neste corpus $P \approx N/2$	$\Theta(V + \sum df)$

- A pontuação linear sempre visita todos os chunks: melhor, pior e médio coincidem. O que varia entre consultas é P, e portanto o custo da ordenação.
- O Insertion Sort só é competitivo com P pequeno ou lista quase ordenada; a varredura entrega os candidatos na ordem do livro, que não é aleatória em relação aos escores, e por isso a medição ficou em 93 % de $P^2/4$ (Seção 12.2).
- O Merge Sort custa $\Theta(P \log P)$ nos três casos, independentemente da distribuição, o que o torna a escolha segura. A medição ficou em 85 % de $P \log_2 P$, porque cada intercalação para quando uma metade se esgota.
- A consulta via índice tem melhor caso quando os termos são raros e degenera ao custo de C3 quando $P \approx N$. É a hipótese de esparsidade da Seção 4.3, e ela falha em consultas em português corrente, onde "que", "de" e "como" aparecem em 15 % a 31 % dos chunks.

10. Recorrências

10.1 Merge Sort

$$T(P) = 2 \cdot T(P/2) + \Theta(P), \quad T(1) = \Theta(1)$$

Origem dos termos na implementação: o fator 2 vem das duas chamadas recursivas sobre `left` e `right`; cada $T(P/2)$ é o custo de ordenar uma metade; o $\Theta(P)$ é o custo de `_merge`, que percorre as duas metades uma única vez e copia cada elemento para a saída. Pela árvore de recorrência, cada um dos $\lceil \log_2 P \rceil + 1$ níveis realiza trabalho $\Theta(P)$, somando $\Theta(P \log P)$. O Teorema Mestre confirma: $a = 2$, $b = 2$, $P^{\log_b a} = P$; como $f(P) = \Theta(P)$, vale o caso 2 e $T(P) = \Theta(P \log P)$. A medição de comparações por dobra de P (razão 2,22 e 2,29) é compatível com esse crescimento.

10.2 Busca binária

$$T(V) = T(V/2) + \Theta(1), \quad T(1) = \Theta(1)$$

Há uma única chamada sobre metade do vocabulário, a outra metade é descartada a cada passo, e o $\Theta(1)$ corresponde ao cálculo do meio e à comparação com `vocab[mid]`. Pelo Teorema Mestre, $a = 1$, $b = 2$, $V^{\log_b a} = 1 = f(V)$: caso 2, $T(V) = \Theta(\log V)$. Na implementação o laço é iterativo, mas a recorrência descreve a mesma redução do intervalo. Com $V = 5.028$, são cerca de 12 comparações por termo; no experimento, a busca binária somou poucas dezenas de comparações por consulta, o que explica a diferença pequena entre C2 e C3 nas comparações totais (13.039 contra 12.746).

11. Metodologia experimental

11.1 Configurações comparadas

Config.	Descrição	Papel
C1, baseline	varredura linear + Insertion Sort (equipe)	busca linear com ordenação mínima
C2, indexada	índice invertido + busca binária no vocabulário + Merge Sort dos candidatos	busca ordenada/indexada
C3, divisão e conquista	varredura linear + Merge Sort (equipe)	otimização da ordenação
C4, referência	tf-idf do scikit-learn com norma L2 e cosseno	baseline externo; não substitui nenhuma exigência

As três configurações obrigatórias usam os mesmos dados e a mesma função de pontuação; diferem só na estrutura de busca e na ordenação. Consequência para a validação: devem produzir listas idênticas, diferindo apenas em custo, o que o script confere a cada bloco (hipótese H1). C4 pode divergir por usar normalização L2 e é analisada à parte.

11.2 Cargas, repetições e execuções

Desenho fatorial: 4 configurações $\times$ 3 tamanhos de corpus (25 %, 50 % e 100 % dos chunks, sorteados por capítulo com semente 2026: 684, 1.368 e 2.737 chunks) $\times$ 30 consultas $\times$ $k \in \{5, 10\} \times 5$ repetições, após 2 execuções de aquecimento descartadas, com a ordem das configurações sorteada em cada bloco. São 3.600 execuções registradas, uma linha por execução em `experimentos/brutos/execucoes.jsonl`; por consulta e valor de k são 45 execuções mensuráveis das configurações obrigatórias, contra o mínimo de 12 do enunciado. O experimento de sensibilidade varia o chunking em 128/16, 256/32 e 512/64.

11.3 Métricas registradas

Por execução: tempo de ingestão (estatísticas N e df), de construção do índice, de busca e pontuação, de ordenação, de consulta e total, com `time.perf_counter_ns`; memória de pico com `tracemalloc` em execução extra, fora do cronômetro; comparações e trocas por contadores internos; tamanho do corpus, número de chunks, candidatos, k , resultados devolvidos e lista devolvida. A partir das listas: Precision@ k , Recall@ k , nDCG@ k e MRR contra o gabarito de relevância; taxa de resultados vazios; fração do top- k na seção esperada. À parte, o tempo real de ingestão (leitura, normalização e fragmentação dos 21 notebooks) por `scripts/medir_ingestao.py`.

11.4 Ambiente e reprodutibilidade

Execução principal em 22/09/2026, das 20h28 às 20h36 (horário de Brasília), num Apple M5 com 24 GB, macOS, Python 3.14.7, scikit-learn 1.6.1, numpy 2.2.2, commit e linha de comando completos em `experimentos/logs/ambiente.json`. Tabelas com mediana e intervalo interquartil por célula, porque um pico do coletor de lixo distorce a média e não a mediana. Tudo roda com `python scripts/reproduzir_tudo.py`; os scripts são idempotentes. A reprodução em máquina limpa (Linux x86_64, 23/09) reproduziu os hashes do corpus e da extração, os 131 testes e as 3.600 execuções com as mesmas comparações, candidatos e listas, mudando apenas os tempos.

11.5 Julgamentos de relevância

As 30 consultas de `data/queries.csv` foram escritas por Helena antes de qualquer resultado ser observado, em três categorias de dificuldade lexical (10 fáceis, 10 médias, 10 difíceis), com capítulo e seção esperados. O gabarito de relevância (`data/perguntas_30_com_chunks_justificativas.csv`) lista, para cada consulta, os chunks que a respondem, de 2 a 8 por consulta, 146 no total, cada um com justificativa, escolhidos no corpus fragmentado sem consultar os rankings; por isso não favorece nenhuma configuração. `scripts/montar_qrels.py` importar confere ids e perguntas e grava `data/qrels.csv`. O julgamento é binário e de uma avaliadora; a escala 0, 1, 2 com dois avaliadores prevista no contrato 03 não foi executada, o que consta como limitação.

12. Resultados e gráficos

12.1 Equivalência e custo por configuração (H1)

C1, C2 e C3 devolveram a mesma lista, na mesma ordem, em todas as 180 combinações de tamanho, consulta e k; o script registrou zero divergências. Nenhuma das 3.600 execuções devolveu lista vazia. A tabela traz as medianas no corpus inteiro, sobre 300 execuções por linha.

Configuração	Total (ms)	Ingestão	Índice	Busca e pontuação	Ordenação	Consulta	Comparações	Memória de pico
C1 varredura + Insertion Sort	167,0	39,4	0	63,2	63,2	127,9	461.702	451 KiB
C2 índice + busca binária + Merge Sort	128,6	39,4	40,8	45,2	2,3	47,7	13.039	1.659 KiB
C3 varredura + Merge Sort	105,7	39,4	0	63,0	2,4	65,8	12.746	452 KiB
C4 scikit-learn (referência)	87,1	39,4	47,2	0,4	(junto com a busca)	0,4	não se aplica	2.581 KiB

Tempos em milissegundos. "Consulta" soma busca, ordenação e corte, sem ingestão e sem índice. A mediana de candidatos com escore > 0 (P) foi 1.403 no corpus inteiro, cerca de metade dos chunks, porque palavras frequentes da consulta como "que", "como" e "uma" aparecem em quase todo trecho.

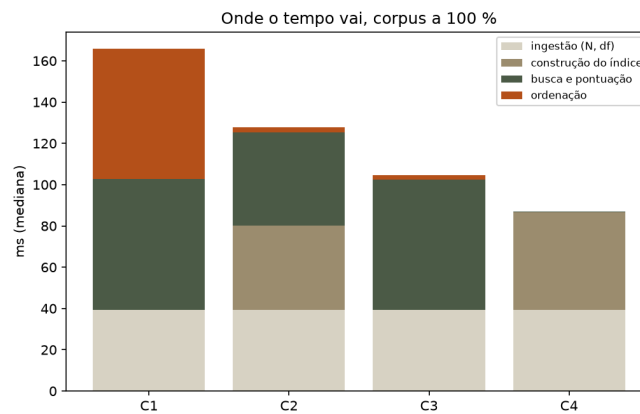


Figura 1. Decomposição do tempo por etapa, corpus inteiro (mediana de 300 execuções por configuração).

12.2 Comparações contra a previsão assintótica (H2)

Chunks (N)	Candidatos (P)	Comparações C1	Razão	Comparações C3	Razão
684	352	29.349		2.512	
1.368	710	115.210	3,93	5.752	2,29
2.737	1.403	461.702	4,01	12.746	2,22

Quando P dobra, as comparações do Insertion Sort multiplicam por cerca de 4, o comportamento de $\Theta(P^2)$, e as do Merge Sort por pouco mais de 2, o de $\Theta(P \log P)$. No corpus inteiro, C1 faz 36 vezes mais comparações que C3 para devolver a mesma lista. Por consulta, o Insertion Sort fez de 85 % a 100 % de $P^2/4$ (mediana 93 %), um pouco abaixo porque a varredura entrega os candidatos na ordem do livro; o Merge Sort fez de 81 % a 88 % de $P \log_2 P$ (mediana 85 %), já que cada intercalação para quando uma metade se esgota. A C2 soma às comparações do Merge Sort as da busca binária, poucas dezenas por consulta.

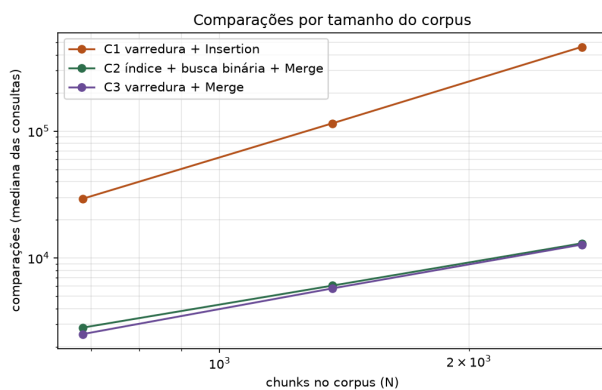


Figura 2. Comparações por tamanho do corpus, eixos logarítmicos: inclinações distintas para P^2 e $P \log P$.

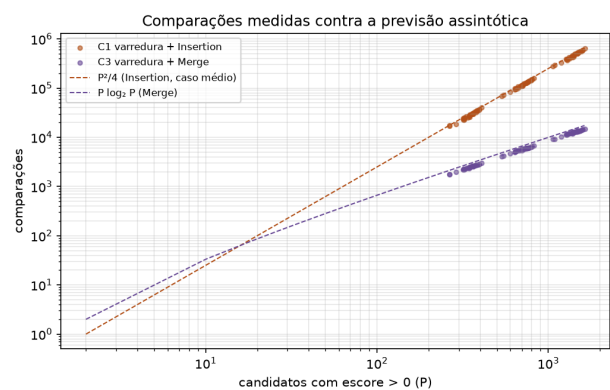


Figura 3. Comparações medidas por consulta sobre as curvas teóricas $P^2/4$ e $P \log_2 P$.

12.3 Tempo e escalabilidade (H3)

Chunks (N)	Total C1 (ms)	Total C2	Total C3	Total C4	Consulta C2	Consulta C3
684	29,0	31,3	25,6	22,0	11,5	16,0

1.368	68,1	65,6	53,4	45,6	24,0	32,9
2.737	167,0	128,6	105,7	87,1	47,7	65,8

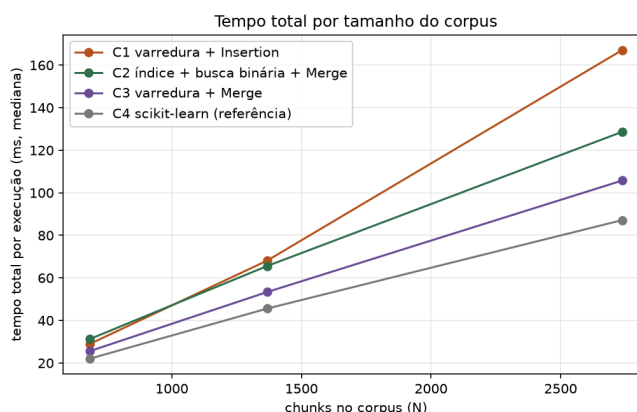


Figura 4. Tempo total por execução em função do número de chunks.

C3, C2 e C4 crescem de forma aproximadamente linear em N . C1 cresce mais (2,35 e 2,45 vezes a cada dobra), porque a fatia da ordenação quadrática aumenta com o corpus: 13 % do total com 684 chunks e 38 % com 2.737. A C2 tem a consulta mais rápida entre as configurações da equipe (47,7 ms contra 65,8 ms), porque pontua só os chunks que contêm algum termo da consulta, mas construir o índice custa 40,8 ms e no experimento ele é reconstruído a cada execução; por isso o total da C2 fica acima do da C3. Com o índice construído uma vez, a C2 passa a sair mais barata que a C3 no acumulado a partir da terceira consulta (40,8 + 47,7n contra 65,8n). O preço é memória: o índice ocupa 3,7 vezes o pico da C3.

12.4 Qualidade da recuperação

Configuração	P@5	P@10	R@10	nDCG@10	MRR	Consultas sem relevante no top-10	Vazios
C1, C2, C3	0,087	0,077	0,158	0,141	0,248	14 de 30	0 %
C4 scikit-learn	0,247	0,183	0,387	0,366	0,551	2 de 30	0 %

C1, C2 e C3 têm exatamente as mesmas métricas, porque devolvem a mesma lista: a escolha do algoritmo define o custo, não a qualidade. A C4 acerta quase três vezes mais no top-5. Por categoria, a vantagem está nas consultas fáceis e médias (P@5 de 0,32 e 0,30 contra 0,12 e 0,04); nas difíceis as duas empatam em nível baixo (0,12 contra 0,10), o limite de qualquer busca lexical quando a pergunta usa palavras diferentes das do livro. A causa da diferença aparece em experimentos/processados/secoes_mais_devolvidas_C1.md: das 300 posições de top-10 da equipe, 110 (36,7 %) são do Glossário, que reúne muitos termos técnicos; a fórmula do contrato 02 soma a contribuição de cada termo sem normalizar pelo tamanho do trecho, e a C4, com norma L2, transforma o escore em cosseno e o Glossário deixa de dominar. A lista da equipe e a da referência têm em comum só 10,7 % do top-5. Itens irrelevantes por consulta: 1 – P@k, ou seja, 91 % do top-5 na equipe e 75 % na C4.

12.5 Ingestão e sensibilidade ao tamanho do chunk

O tempo real de ingestão (leitura dos 21 notebooks, extração e normalização das células e fragmentação) foi medido por `scripts/medir_ingestao.py`, 5 medidas após 2 aquecimentos, numa máquina diferente da do experimento principal (Linux x86_64, 4 núcleos), então os tempos valem pela proporção entre linhas; candidatos, comparações e o indicador de seção não dependem da máquina.

tamanho	sobreposicao	n_chunks	tokens_mediana	tokens_max	chunks_de_celula_cortada_%	extracao_ms	fragmentacao_ms	ingestao_ms	ing
128	16	2821	14	128	6.2	53.13	86.27	138.76	4.05
256	32	2737	13	256	0.8	55.21	82.92	137.78	7.85
512	64	2726	13	388	0.0	51.16	81.03	131.97	2.14

O tamanho da janela quase não muda o corpus, porque o corte respeita a célula e a mediana das células fica em 13 tokens: o número de chunks varia 3,5 % entre 512 e 128 tokens, e a ingestão fica em torno de 135 ms, dominada pela fragmentação. O efeito aparece onde a análise prevê: com 128 tokens, P cresce 5,8 % e as comparações do Insertion Sort crescem 11,5 %, perto do quadrado de 1,058 (11,9 %); as do Merge Sort, 6,6 %. O indicador de qualidade sobe de 7,3 % para 10 % com janelas menores, porque cortar o Glossário reduz a vantagem dos trechos longos (as posições de top-10 do Glossário caem de 110 para 89 das 300). A linha 256/32 reproduz os números do experimento principal, o que confere os dois scripts um contra o outro.

13. Discussão de escalabilidade

Para o corpus atual as três configurações rodam em dezenas a centenas de milissegundos, e a tendência é o que interessa. Extrapolando as razões medidas: com um livro quatro vezes maior, a ordenação de C1 passaria de 1 s por consulta, enquanto a de C3 ficaria perto de 10 ms. C1 torna-se inviável em acervos grandes porque o Insertion Sort cresce como P^2 e a varredura toca todos os N chunks. C3 mantém a varredura $\Theta(N \cdot m + L)$ por consulta, mas a ordenação passa a ser 2 % do tempo; o gargalo vira a pontuação, que numa aplicação real seria feita uma vez, junto com as estatísticas do corpus.

A previsão de que C2 desacoplaria o custo da consulta do tamanho do corpus dependia da hipótese de esparsidade, e os dados a refutaram: os candidatos são 51 % dos chunks (de 39 % a 60 % conforme a consulta), porque as consultas em português corrente carregam palavras como "que" (29 % dos chunks), "de" (31 %) e "como" (15 %). Com $P \approx N/2$, o custo de C2 continua crescendo com N , e o ganho na consulta foi de 1,4 vez sobre C3, não de ordens de grandeza. Ainda assim o índice é a única das três que evita visitar todos os chunks, e o ganho crescerá com remoção de stopwords ou com listas de postings mais seletivas; o trade-off clássico permanece: gastar memória e tempo de indexação uma vez (40,8 ms e 3,7 vezes a memória) para tornar cada consulta mais barata, o que compensa a

partir da terceira consulta sobre o mesmo corpus. Bibliotecas como FAISS levam a mesma ideia ao caso vetorial denso; aqui ela foi demonstrada no caso lexical, com estruturas clássicas e medida contra a hipótese que a sustenta.

14. Relação com IA generativa e RAG

O contexto recuperado pelo protótipo é a etapa de *retrieval* de um sistema RAG. Sobre os sete pontos pedidos:

1. **Incorporação ao prompt.** Os k chunks devolvidos seriam concatenados em um bloco de contexto ("Contexto: {chunks} / Pergunta: {q}"), cada trecho acompanhado da origem (notebook, seção) para citação. Com $k = 5$ e chunks de mediana 13 tokens, o bloco é pequeno; com trechos longos como o Glossário, cresce várias vezes.
2. **Impacto de k , tamanho de chunk e ordenação no custo de contexto.** Mais chunks e chunks maiores gastam tokens da janela do LLM, aumentando latência e custo de geração. A ordem importa: modelos tendem a subaproveitar informação no meio do contexto (*lost in the middle*), e a ordem total $\leq$ garante que o trecho de maior score venha primeiro. O experimento de sensibilidade mostra que chunks menores custam um pouco mais de ordenação e melhoram levemente a qualidade.
3. **Lexical contra semântico.** O tf-idf captura sobreposição de termos, não sinônimos nem paráfrases: "laço" não casa com "loop". As 14 consultas sem nenhum trecho relevante no top-10 e o empate em nível baixo nas consultas difíceis mostram esse limite; busca semântica com embeddings recuperaria trechos que a lexical não alcança.
4. **Riscos de recuperação incompleta, enviesada, desatualizada ou irrelevante.** Incompleta: Recall@10 de 0,158 na equipe. Enviesada: um único livro cobre uma visão, e a fórmula sem normalização favorece o Glossário. Desatualizada: o commit fixo controla a desatualização, mas a torna real. Irrelevante: 91 % do top-5 da equipe não está no gabarito.
5. **Afirmações não sustentadas.** Mesmo com bom contexto, o LLM pode alucinar além do recuperado; com contexto irrelevante, o risco cresce. Mitiga-se com instrução explícita de responder só com base no contexto e citar a origem de cada afirmação, e com a rastreabilidade do item seguinte.
6. **Rastreabilidade.** Cada chunk carrega a proveniência (notebook, seção, célula, offsets), então a resposta pode citar a fonte exata e o usuário confere no livro, uma melhoria direta de confiabilidade sobre um LLM sem recuperação.
7. **Trade-offs.** Precisão contra tempo, memória e custo de geração: C4 triplica a precisão e tem a consulta mais rápida, com 6 vezes a memória de C3; C2 acelera a consulta em 1,4 vez ao preço de um índice com 3,7 vezes a memória; aumentar k eleva o recall potencial e também o custo em tokens e o ruído.

15. Limitações e ameaças à validade

- **Validade de construção.** A Precision@ k depende de julgamentos de relevância: o gabarito foi montado antes dos rankings e com justificativas, mas é binário, de uma só avaliadora, e sem a adjudicação prevista no contrato 03. O indicador de seção esperada confirma a mesma tendência.
- **Validade interna.** O cronômetro mede o interpretador Python tanto quanto o algoritmo; por isso medianas de cinco repetições, aquecimento descartado, ordem sorteada, e os contadores de comparações como evidência principal. A memória vem do `tracemalloc`, não do RSS do processo.
- **Validade externa.** Um livro introdutório em português, do domínio de programação, é um estudo de caso, não uma amostra dos materiais educacionais abertos. Os três tamanhos são subconjuntos do mesmo livro e P cresce junto com N , o que mistura os dois efeitos; três tamanhos mostram tendência, não provam ordem assintótica.
- **Viés do corpus e do chunking.** A tradução mistura português e termos em inglês. Os chunks seguem as células dos notebooks (mediana de 13 tokens), não janelas contínuas de 256, o que multiplica trechos curtos e deixa o Glossário como trecho longo dominante. A dificuldade das consultas coincide com a ordem dos capítulos, confundindo dificuldade lexical com assunto.
- **Hipótese de esparsidade.** Declarada na definição do problema e refutada pelos dados: metade dos chunks vira candidato. É a principal lição de projeto do trabalho.
- **Validade de conclusão.** Os tempos vêm de um único ambiente; a reprodução em máquina limpa manteve as comparações e listas idênticas e mudou só os tempos, por um fator uniforme de 2,1.

16. Declaração de uso de IA generativa

Reprodução de docs/USO_DE_IA.md, preenchido durante o trabalho. As linhas "a preencher" correspondem a integrantes que não registraram uso até a entrega.

Este arquivo é preenchido durante o trabalho, não na véspera da entrega. Cada uso relevante entra aqui no dia em que aconteceu, com o que foi verificado e por quem.

Ferramentas e modelos

Ferramenta	Modelo exibido na interface	Período de uso
OpenAI Codex (João)	conferir na interface	01/09/2026
Claude Code, Anthropic (João)	Claude Fable 5.1; Claude Opus 5.5	02/09/2026 a 24/09/2026
ChatGPT, OpenAI (João)	GPT-6 Astra	setembro de 2026
ChatGPT, OpenAI (Nadianne)	GPT-5.6 Sol	18/09/2026 a 19/09/2026
a preencher	a preencher	a preencher

Finalidade de cada uso

Data	Integrante	Finalidade	Onde entrou no trabalho
01/09	João	Rascunho do plano de 19 seções a partir do enunciado	Planejamento interno, não versionado
02/09 a 04/09	João	Estrutura do repositório, script de aquisição com manifesto de hashes, contratos técnicos e divisão de frentes	README.md, scripts/baixar_corpus.py, docs/CONTRATOS.md, docs/SEMANA1.md
14/09 a 16/09	João	Fixtures sintéticos com gabarito, fragmentação, índice invertido e revisão do código de pontuação e desempate	tests/fixtures/, src/paa_context/fragmentacao.py, indice.py, relevance.py, sort_key.py
19/09	João	Medição dos contadores contra a previsão assintótica e teste de mutação do Merge Sort, em cópia descartável	Números usados na análise e em tests/README.md
22/09	João	Conversão das 30 perguntas para data/queries.csv, com as seções conferidas contra o corpus, e redação de tests/README.md	data/queries.csv, docs/CONTRATOS.md, tests/README.md
23/09	João	Levantamento das pendências da entrega, reprodução completa em máquina limpa, script de ingestão e sensibilidade ao tamanho do chunk, atualização da documentação	scripts/medir_ingestao.py, experimentos/processados/ingestao_e_sensibilidade.md, docs/RESULTADOS.md, data/README.md, tests/README.md
23/09 a 24/09	João	Conferência do relatório contra o código e o enunciado, importação do gabarito de relevância para qrels.csv, roteiro do vídeo, apresentação em slides gerada a partir dos resultados	scripts/montar_qrels.py, data/qrels.csv, slides/apresentacao_av1.pdf, VIDEO.md
18/09 a 19/09	Nadianne	Apoio na análise formal de corretude do Merge Sort, definição das hipóteses, prova por indução, invariante de _merge, término e limites do argumento	docs/CORRETUDE.md
18/09 a 19/09	Nadianne	Apoio na análise no modelo RAM, melhor/pior/médio caso e recorrências dos algoritmos	documento de análise em docs/ e slides
19/09	Nadianne	Apoio na elaboração de testes específicos da relação de ordenação por score e source_order	tests/unit/test_sort_key.py
a preencher	a preencher	a preencher	a preencher

Prompts relevantes

O enunciado admite até cinco. Registrar o texto real enviado, não uma reconstrução aproximada.

- (João, 22/09) "Você analisou o [repositório] preciso saber o que realmente interfere na nota, pois não quero ficar sendo o chato pedindo pra o pessoal corrigir"
- (João, 22/09) "converte o PDF para o queries.csv, veja o que mais posso fazer pelo grupo"
- (Nadianne, 19/09) "Como ligar esse trecho e a análise teórica as configurações C1, C2 e C3?"
- a preencher
- a preencher

Sugestões aproveitadas

Sugestão	Por que foi aceita	Quem validou
Postings como listas de posições crescentes e vocabulário ordenado, para que a busca binária tenha papel real na C2	Com dicionário na consulta a busca binária seria decorativa	João, com os testes de test_indice.py e a equivalência C1 = C2 = C3
Registrar secao_esperada com o nome exato da seção no corpus	Permite montar o julgamento de relevância a partir das seções	João, conferindo as 30 linhas contra data/processed/chunks.jsonl
Estruturar a prova do Merge Sort com hipótese, indução, invariante e término	A estrutura corresponde ao comportamento recursivo da implementação e à metodologia estudada na disciplina	Nadianne, comparando a análise com merge_sort.py, sort_key.py e os testes existentes

Sugestão	Por que foi aceita	Quem validou
Criar testes específicos para <code>sort_key</code> , cobrindo maior <code>score</code> , desempate por <code>source_order</code> e transitividade	As propriedades são usadas diretamente como hipóteses na prova formal	Nadianne, executando <code>test_sort_key.py</code> e a suíte completa com 118 testes aprovados
a preencher	a preencher	a preencher

Sugestões corrigidas ou rejeitadas

Sugestão	Problema identificado	O que a equipe fez no lugar
Integração contínua no GitHub como item obrigatório	O enunciado não exige CI	Mantida fora do escopo
Numeração do plano inicial: C2 = Merge Sort, C3 = índice	Invertida em relação ao enunciado	Adotada a numeração do professor em <code>docs/CONTRATOS.md</code>
Descrever o comportamento de empate apenas como “estabilidade” do Merge Sort	A ordem entre candidatos com mesmo <code>score</code> não depende simplesmente da ordem de entrada, há um critério explícito de desempate por <code>source_order</code>	A análise passou a usar “desempate determinístico por <code>source_order</code> ”, mantendo estabilidade clássica separada desse conceito
a preencher	a preencher	a preencher

Erros identificados

Registrar erros concretos de correteude, complexidade, casos de borda, estabilidade de ordenação ou desempate que a equipe encontrou em saída de IA. Este é o item que o enunciado cobra com mais rigor, porque distingue uso crítico de uso passivo.

Erro	Como foi detectado	Correção aplicada
(João) Na revisão da análise de complexidade, a fórmula da C2 não trazia o termo $\Theta(P(m + n))$ da pontuação dos candidatos	Medição do tempo por etapa em 19/09, feita com apoio de IA: a pontuação é 96% do tempo da consulta e a ordenação, 4%	Termo apontado para inclusão no relatório; os números estão em <code>docs/RESULTADOS.md</code>
Plano de 01/09 com as configurações C2 e C3 trocadas	Conferência contra o PDF do enunciado em 04/09	Numeração corrigida antes de qualquer código
(Nadianne) Tratamento inicial do desempate como simples estabilidade da ordenação	Conferência da implementação de <code>sort_key</code> , que usa explicitamente <code>(-score, source_order)</code>	A prova passou a descrever o comportamento como desempate determinístico por <code>source_order</code>
a preencher	a preencher	a preencher

Verificação

Descrever os testes, provas, execuções e revisões por pares usados para validar cada trecho aproveitado. Código, prova ou análise sugerida por IA e não verificada não entra no trabalho.

João: cada módulo entrou com testes próprios (131 no total em 23/09, `python -m pytest`). O gabarito dos fixtures foi calculado à parte em `tests/fixtures/calcular_gabarito.py` e conferido à mão para um `escore`. A equivalência $C1 = C2 = C3$ é testada em `tests/integration/test_equivalencia.py`. Os contadores de comparações foram confrontados com a previsão assintótica (Merge Sort a 1% de $P \log_2 P$; Insertion Sort a 9% de $P^2/4$). O teste de mutação de 19/09 mostrou um limite da suíte, registrado em `tests/README.md`.

Nadianne: a análise formal foi conferida contra a implementação de `merge_sort.py` e `sort_key.py`. Foram adicionados três testes em `tests/unit/test_sort_key.py`, verificando prioridade por maior `score`, desempate por menor `source_order` e um caso de transitividade. Os testes específicos passaram (3 `passed`) e, em seguida, a suíte completa foi executada com 118 `passed`, sem regressões.

Contribuição individual no uso de IA

Integrante	O que fez com apoio de IA	O que fez sem apoio de IA
João Cosme Sena Sá	Estrutura do repositório, fixtures, fragmentação, índice invertido, conversão das consultas, documentação de testes	Decisões de escopo e divisão de frentes, revisão de cada trecho antes do commit, execução dos testes e das medições
Nadianne Maria dos Santos Galvão	Estruturação e revisão textual da prova de correteude; apoio na análise RAM, complexidade e recorrências, elaboração inicial dos testes de <code>sort_key</code>	Conferência da análise com o código da equipe, execução e validação dos testes, revisão das hipóteses da prova, adaptação do conteúdo ao trabalho e gravação da própria participação no vídeo
a preencher	a preencher	a preencher

17. Contribuição individual

O enunciado pede contribuição com evidência, não cargo. Reprodução de `docs/CONTRIBUICOES.md`: cada linha aponta commits, arquivos, análises ou execuções que comprovam o trabalho descrito. As linhas "a preencher" correspondem a integrantes que não registraram evidências até a entrega. A revisão de código, da prova e dos resultados é responsabilidade coletiva.

Integrante	Frete	Evidências	Participação no vídeo
João Cosme Sena Sá	Dados de teste, pacote e testes, protocolo experimental, índice invertido e busca binária (C2), entrega	Estrutura do repositório e aquisição do corpus com manifesto de hashes (60447c2 , 766e210); contratos técnicos e plano da semana 1 (5f25ff0); <code>pyproject.toml</code> e configuração do <code>pytest</code> (ebf7698); modelo do chunk e fixtures sintéticos com gabarito calculado (8fab1d3 , 46f436c , 4ecf106 , 56ffd8f); busca binária (3a0fe46); fragmentação dos notebooks e <code>preparar_corpus.py</code> , assumidas como apoio à frente de ingestão (2926971 , 2f14b3a); <code>consultar.py</code> com C1 e C3 (663e767); protocolo experimental (2e85a1a); pontuação tf-idf, desempate por <code>source_order</code> e ordenação só dos candidatos (4370ad2); índice invertido e C2 integrada (91b4466 , d18ce80); <code>data/queries.csv</code> a partir das 30 perguntas da equipe; <code>tests/README.md</code> ; pool de julgamento de relevância (<code>scripts/montar_qrels.py</code>); baseline de referência C4, métricas de qualidade, <code>rodar_experimentos.py</code> , <code>gerar_figuras.py</code> e <code>reproduzir_tudo.py</code> , assumidos na ausência da frente de experimentos (b8498d5); execução do experimento principal, 3.600 execuções, com tabelas e figuras (6cfbcd4); análise dos resultados em <code>docs/RESULTADOS.md</code>	Abertura da equipe, tema e corpus; índice invertido e busca binária (C2) no código; demonstração das três configurações com a mesma consulta; lição e limite dos resultados; gravação individual: https://www.loom.com/share/aab4b4cd6a934bb892fd6c4bb73bcd85
Eduardo Henrique do Lago Silva	Experimentos, tabelas, gráfico e baseline de referência (C4)	a preencher	a preencher
Helena Carvalho Leal	Consultas, julgamentos de relevância, métricas, relação com IA generativa e limitações	Elaboração e revisão das 30 consultas de avaliação (Q01 – Q30), organizadas em 10 fáceis, 10 médias e 10 difíceis e alinhadas ao corpus da 3ª edição em português de <i>Pense Python</i> ; definição dos temas, palavras-chave e capítulos/seções esperados para cada consulta; preenchimento manual do CSV de gabarito de relevância, associando as consultas aos <code>chunk_id</code> relevantes do corpus fragmentado (<code>chunks.jsonl</code>) antes da avaliação dos rankings.	a preencher
Hernandison da Silva Bispo	Ingestão, normalização e fragmentação; manutenção do relatório	a preencher	a preencher
Nadianne Maria dos Santos Galvão	Corretude, modelo RAM, complexidade e recorrências	<code>docs/CORRETUDE.md</code> ; análise RAM/complexidade/recorrências em <code>docs/</code> ; criação de <code>tests/unit/test_sort_key.py</code> com testes de prioridade por <code>score</code> , desempate por <code>source_order</code> e transitividade; suíte completa com 118 testes aprovados; commit 6a5923c e PR da branch <code>Nadianne_01</code>	Apresentação de RAM e complexidade, recorrências/C1-C2-C3 e corretude formal do Merge Sort; gravação individual da participação no https://drive.google.com/file/d/1WWHyEl1k9qLuE8Sod03WE9qJ3bNDWx3Y/view?usp=drive_link
Rafael Takeguma Goto	Busca linear, Insertion Sort, Merge Sort, pontuação e consulta (C1 e C3)	Pré-processamento de texto (<code>preprocessing.py</code>) e primeira versão da função de relevância (<code>relevance.py</code>), com testes unitários (9ae619b); busca linear sobre os chunks (<code>linear_search.py</code> , <code>test_linear_search.py</code> , edc590d); Insertion Sort (<code>insertion_sort.py</code> , <code>test_insertion_sort.py</code> , 6f025c0); Merge Sort (<code>merge_sort.py</code> , <code>test_merge_sort.py</code> , 3338455); contador de operações elementares (<code>contador.py</code>), seleção dos top-k, critério de desempate nas ordenações e pipeline de consulta (<code>pipeline.py</code> , <code>test_pipeline.py</code>) (f660124); merge do PR #1 (branch <code>Nadianne_01</code> , c1ca7a2)	https://drive.google.com/file/d/13jiWmtcPA6ZGqowYDyxqEHd5966waPa8/view?usp=sharing

18. Vídeo da atividade

O vídeo tem até 10 minutos, com participação de todos os integrantes e identificação da contribuição de cada um. Até o fechamento deste relatório as gravações estão publicadas individualmente, com reprodução liberada por link:

- João Cosme Sena Sá: <https://www.loom.com/share/aab4b4cd6a934bb892fd6c4bb73bcd85> (abertura, C2 no código, demonstração das três configurações, lição e limite)
- Nadianne Maria dos Santos Galvão: https://drive.google.com/file/d/1WWhyE11k9qLuE8Sod03WE9qJ3bNDWx3Y/view?usp=drive_link (modelo RAM, complexidade, recorrências e corretude do Merge Sort)
- Rafael Takeguma Goto: <https://drive.google.com/file/d/13jiWmtcPA6ZGqowYDyxqEHd5966waPa8/view?usp=sharing> (busca linear, Insertion Sort, Merge Sort, pontuação e consulta)

A URL do vídeo consolidado substitui esta lista no `README.md` (seção "Vídeo da atividade"), no `VIDEO0.md` (com data de gravação e participantes) e na capa deste relatório quando a junção for publicada.

19. Referências

1. CORMEN, T. H.; LEISERSON, C. E.; RIVEST, R. L.; STEIN, C. *Introduction to Algorithms*. 4. ed. Cambridge: MIT Press, 2022.
2. KLEINBERG, J.; TARDOS, É. *Algorithm Design*. Boston: Pearson, 2006.
3. SKIENA, S. S. *The Algorithm Design Manual*. 3. ed. Cham: Springer, 2020.
4. SEDGEWICK, R.; WAYNE, K. *Algorithms*. 4. ed. Boston: Addison-Wesley, 2011.
5. MANNING, C. D.; RAGHAVAN, P.; SCHÜTZE, H. *Introduction to Information Retrieval*. Cambridge University Press, 2008.
6. LEWIS, P. et al. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. *NeurIPS*, 2020. arXiv:2005.11401.
7. ZHOU, Z. et al. A Survey on Efficient Inference for Large Language Models. arXiv:2404.14294, 2024.
8. GAN, A. et al. Retrieval Augmented Generation Evaluation in the Era of Large Language Models: A Comprehensive Survey. arXiv:2504.14891, 2025.
9. PEDREGOSA, F. et al. Scikit-learn: Machine Learning in Python. *JMLR*, v. 12, 2011.
10. FAISS. *Faiss documentation*. <https://faiss.ai/>.
11. DOWNEY, A. B. *Think Python*. 3. ed. O'Reilly, 2024. Tradução livre *Pense Python* por CARLSON, R. C. <https://rodrigocarlson.github.io/PensePython3ed/>.
12. CARNEIRO, G. F. *Projeto e Análise de Algoritmos: Atividade 1 (AV1)* e notas de aula 02 a 05, PROCC/UFS, 2026.2.